Observability

Instrument systems with metrics, logs and traces, ship them with OpenTelemetry, probe from outside, and alert on what users actually feel.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Observability fundamentals (13 min)
2. Metrics and Prometheus (17 min)
3. Logs and distributed tracing (15 min)
4. Log pipelines with Loki, Elasticsearch and Fluent Bit (15 min)
5. Instrumenting with OpenTelemetry (16 min)
6. Blackbox probes and synthetic monitoring (14 min)
7. Alerting and dashboards (16 min)
8. RED, USE and the golden signals in practice (13 min)
9. Telemetry cost, cardinality and retention (13 min)
10. Structured logging and log design (12 min)
11. Dashboards and runbooks for on-call (12 min)
12. Debugging with metrics, logs and traces (15 min)

Lesson 1. Observability fundamentals

13 minutes. Learn the difference between monitoring and observability, the three signals of metrics, logs and traces, and the RED and USE methods for deciding what to measure.

A customer reports that checkout is slow. You open a dashboard full of graphs, all green. The system is "up" and yet users are suffering. Observability is the ability to ask new questions of a running system and get answers without shipping new code. It goes beyond watching a list of known failure signs.

What you will learn

- Monitoring versus observability
- Metrics, logs and traces, and what each is good for
- The RED and USE methods
- Choosing what to measure first

Monitoring versus observability

Monitoring watches for failures you anticipated: disk above 90%, process down, error rate above a threshold. **Observability** is a property of the system: it emits enough detailed data that you can investigate failures you did not anticipate. You need both. Monitoring tells you something is wrong; observability helps you find out why.

The three signals

Signal	What it is	Strength	Weakness
Metrics	Numbers sampled over time (requests per second, CPU)	Cheap, fast to query, ideal for alerts and trends	Aggregated, so individual requests are invisible
Logs	Timestamped records of events	Detailed context for a single event	Expensive at volume, harder to aggregate
Traces	The path of one request through many services, with timing for each step	Shows where time is spent and which dependency failed	Needs instrumentation across services

They work together. A metric alert fires on high latency; a trace shows the slow call is to the payment service; the log lines from that service for the same request ID show a timeout to its database.

> **Tip:** *Put a* **request ID** *in every log line and pass it between services. It is the cheapest way to connect the three signals.*

What to measure: RED and USE

Two checklists save you from guessing.

RED is for request-driven services (APIs, web apps):

Letter	Measure
Rate	Requests per second
Errors	Failed requests per second, or the error ratio
Duration	How long requests take, as percentiles (p50, p95, p99)

USE is for resources (CPUs, disks, network interfaces, queues):

Letter	Measure
Utilization	How busy the resource is
Saturation	How much work is waiting (queue length, load)
Errors	Error counts (disk errors, dropped packets)

MARGA by EknathaLabs marga.eknathalabs.com

Use **percentiles, not averages**, for latency. An average of 200 ms can hide that 1% of requests take 8 seconds. The slowest users are often your most valuable ones, because they do the most.

The golden signals

Google's SRE book names four signals that fit nearly any service: latency, traffic, errors and saturation. They are RED plus saturation, and they make a good first dashboard for anything you run.

Instrumentation and standards

OpenTelemetry is the vendor-neutral standard for generating metrics, logs and traces from applications. Instrument once with it and send the data to whichever backend you choose, such as Prometheus for metrics, a log store, and a tracing system. This avoids rewriting application code if you change tools.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Observability fundamentals fits together

Part	What it does	What goes wrong
Monitoring versus observability	Monitoring watches for failures you anticipated: disk above 90%, process down, error rate above a threshold.	Check the Common mistakes list below this diagram for what usually goes wrong here.
The three signals	They work together. A metric alert fires on high latency; a trace shows the slow call is to the payment service; the log lines from that service for the same request ID...	Dashboards with 40 graphs and no clear question they answer.
What to measure: RED and USE	Two checklists save you from guessing.	Using averages for latency.
The golden signals	Google's SRE book names four signals that fit nearly any service: latency, traffic, errors and saturation.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Instrumentation and standards	OpenTelemetry is the vendor-neutral standard for generating metrics, logs and traces from applications.	Logging everything with no structure, so nothing can be queried.

Flow: Monitoring versus observability to The three signals / The three signals to What to measure: RED and USE / What to measure: RED and USE to The golden signals / The golden signals to Instrumentation and standards

Incident: all green, users unhappy

> **Incident:** *Customers complain about slow pages. The dashboard shows CPU at 30%, memory fine, and an average response time of 180 ms.*

The dashboard tracked resource use and an average, so it missed the problem. Re-ask the question with percentiles:

```
histogram_quantile(0.99, sum by (le, route) (rate(http_request_duration_seconds_bucket[5m])))
```

This sample query shows the 99th percentile per route. The `/checkout` route is at 6.4 seconds while others are under 300 ms. A trace of one slow checkout request shows 6 seconds spent waiting on a single call to the inventory service, whose database connection pool is exhausted. None of that was visible in averages or CPU. The fix belongs to that service (pool size and a timeout), and the observability fix is to add per-route latency percentiles and a saturation metric for connection pools to the standard dashboard so the next such case is visible in a minute.

Common mistakes

- Alerting on causes (CPU high) instead of symptoms users feel
- Using averages for latency

MARGA by EknathaLabs marga.eknathalabs.com

- Logging everything with no structure, so nothing can be queried
- Dashboards with 40 graphs and no clear question they answer

Check yourself

Which signal is best for knowing where time was spent across several services for one request?

Answer: Traces

What do the letters in RED stand for?

Answer: Rate, Errors, Duration

Why use p99 latency rather than the average?

Answer: Averages hide slow requests affecting a small share of users

What is the cheapest way to connect logs, metrics and traces?

Answer: A request ID included in every log line and passed between services

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Metrics and Prometheus

17 minutes. Understand the Prometheus data model, scrape configuration and the PromQL queries you will use daily, including rates, aggregation and histograms.

Prometheus is the most widely used open source system for metrics, and its query language, PromQL, is a skill that transfers to many other tools. The model is simple: it **pulls** numeric time series from your services at a regular interval, stores them, and lets you query them.

What you will learn

- Metric types and labels
- Scrape configuration
- Essential PromQL: rate, sum by, histogram_quantile
- Cardinality, the way Prometheus gets expensive

Metric types

Type	Behaviour	Example
Counter	Only goes up (resets on restart)	`http_requests_total`
Gauge	Goes up and down	`memory_usage_bytes`, `queue_length`
Histogram	Counts observations in buckets, so you can compute percentiles	`http_request_duration_seconds_bucket`
Summary	Pre-computed quantiles in the client	Less used; cannot be aggregated across instances

Each series is identified by a metric name plus **labels**:

```
http_requests_total{service="shop-api", route="/checkout", status="500"} 1027
```

Labels make a metric slice-able: by service, route, status. A metric exposed by an application appears on an HTTP endpoint (conventionally `/metrics`) in a simple text format.

Scraping

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: shop-api
    metrics_path: /metrics
    static_configs:
      - targets: ["shop-api-1:8080", "shop-api-2:8080"]

  - job_name: node
    static_configs:
      - targets: ["node-1:9100", "node-2:9100"]
```

In Kubernetes you normally use **service discovery** so Prometheus finds Pods automatically, usually through an operator that adds `ServiceMonitor` objects. Every scraped target gets an automatic `up` metric: `1` if the last scrape worked, `0` if not. Alert on `up == 0`.

MARGA by EknathaLabs
marga.eknathalabs.com

PromQL you will use constantly

```
# Requests per second, per route, averaged over the last 5 minutes
sum by (route) (rate(http_requests_total[5m]))

# Error ratio: share of requests that are 5xx
sum(rate(http_requests_total{status=~"5.."}[5m]))
  /
sum(rate(http_requests_total[5m]))

# 95th percentile latency per route
histogram_quantile(0.95,
  sum by (le, route) (rate(http_request_duration_seconds_bucket[5m])))

# Memory used per node as a percentage
100 * (1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)

# Disk predicted to fill within 4 hours
predict_linear(node_filesystem_avail_bytes{mountpoint="/"}[1h], 4 * 3600) < 0
```

Key rules:

- **Never graph a raw counter.** Wrap it in `rate()` (per-second increase) or `increase()` (total over the range). A raw counter only climbs.
- Choose a `rate()` window of **at least four times the scrape interval** (so 1m for 15s scrapes, and 5m is a common default).
- `sum by (label)` keeps only the labels you name; `without (label)` removes the ones you name.
- `histogram_quantile` needs the `le` label preserved in the aggregation.
- Average percentiles are meaningless. Aggregate the buckets first, then compute the quantile.

Recording rules

Expensive queries that you use often can be precomputed:

```
groups:
  - name: shop-api.rules
    rules:
      - record: route:http_requests:rate5m
        expr: sum by (route) (rate(http_requests_total[5m]))
```

Name them `level:metric:operation`, and dashboards and alerts become fast and consistent.

Cardinality

Each unique combination of label values is a separate time series. Labels with unbounded values, such as `user_id`, `email` or a full URL with IDs, create millions of series and can exhaust memory.

> **Warning:** *Never use user IDs, request IDs, IP addresses or raw URLs as metric labels. Those belong in logs and traces. Label values should come from a small, known set.*

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Metrics and Prometheus fits together

Part	What it does	What goes wrong
Metric types	Each series is identified by a metric name plus labels.	Averaging percentiles across instances.
Scraping	In Kubernetes you normally use service discovery so Prometheus finds Pods automatically, usually through an operator that adds ServiceMonitor objects.	Not alerting on up == 0, so a dead target silently stops reporting.
PromQL you will use constantly	Key rules.	A rate() window shorter than two scrapes.
Recording rules	Expensive queries that you use often can be precomputed.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Cardinality	Each unique combination of label values is a separate time series.	High-cardinality labels.

Flow: Metric types to Scraping / Scraping to PromQL you will use constantly / PromQL you will use constantly to Recording rules / Recording rules to Cardinality

Incident: Prometheus runs out of memory

Incident: *Prometheus restarts repeatedly with out-of-memory kills, and its dashboards have gaps.*

Find what is generating the series:

```
topk(10, count by (__name__) ({__name__=~".+"}))
```

```
$ curl -s localhost:9090/api/v1/status/tsdb | jq '.data.seriesCountByMetricName[0:3]'
[
  { "name": "http_requests_total", "value": 4812377 },
  ...
]
```

Nearly five million series come from `http_requests_total`. A recent release added a `path` label containing the raw URL, including order numbers (`/orders/48213`). Every order created a new series. Remove the label (or normalise it to the route template `/orders/:id`) in the application, drop the label at scrape time with `metric_relabel_configs` as a stop-gap, and restart. Add an alert on `prometheus_tsdb_head_series` growth and a review check for new labels, because the problem arrived with a code change and will return with another.

Predict the query

Common mistakes

- Graphing raw counters
- Averaging percentiles across instances
- High-cardinality labels
- A `rate()` window shorter than two scrapes
- Not alerting on `up == 0`, so a dead target silently stops reporting

Check yourself

Which metric type should you wrap in rate() to see requests per second?

Answer: Counter

Why is a label like user_id dangerous on a metric?

Answer: Each distinct value creates a new time series, causing memory exhaustion

What does the up metric show?

Answer: Whether the last scrape of the target succeeded

MARGA by EknathaLabs marga.eknathalabs.com

To compute a p95 across many instances, what should you do?

Answer: Sum the bucket rates by le first, then apply histogram_quantile

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Logs and distributed tracing

15 minutes. Write structured logs that can be queried, follow requests across services with traces, and keep telemetry volume and cost under control.

Metrics tell you that something is wrong. Logs and traces tell you what happened and where. Their value depends almost entirely on how they are produced: a clear structure and a shared request ID turn a pile of text into an investigation tool.

What you will learn

- Structured logging and log levels
- What to log and what never to log
- How traces, spans and context propagation work
- Sampling and retention

Structured logs

Plain text logs are written for people and are hard for machines to search. **Structured logs** emit one JSON object per event, with fields you can filter on.

```
{"ts":"2026-10-24T03:12:01Z","level":"error","service":"shop-api","request_id":"c7d1e2","route":"/ \
    checkout","user_tier":"premium","duration_ms":6402,"error":"timeout calling inventory","depend \
    ency":"inventory"}
```

Now a log system can answer "show errors for `/checkout` where `dependency` is `inventory`" precisely. Good fields to include: timestamp in UTC, level, service name, version, request ID, and relevant business context.

Level	Use
`error`	A failure needing attention
`warn`	Unexpected but handled
`info`	Significant normal events (start-up, a request summary)
`debug`	Detail for diagnosis, off by default in production

Write logs to **standard output** and let the platform collect them. In containers, applications should not manage log files or rotation themselves.

> **Warning:** *Never log passwords, tokens, full card numbers, or personal data you do not need. Logs are copied to many systems, kept for long periods and read by many people. Mask or omit sensitive fields at the source.*

Searching logs

Whatever the backend (Loki, Elasticsearch, a cloud log service), the workflow is the same: narrow by time, then service, then a field, then follow an ID.

```
$ kubectl logs deploy/shop-api -n shop --since=15m | grep '"level":"error"' | head -3
$ kubectl logs deploy/shop-api -n shop --since=15m | jq -c 'select(.request_id=="c7d1e2")'
```

In Loki's LogQL, a similar search over a time window is:

```
{app="shop-api"} | json | level="error" | route="/checkout"
```

Distributed tracing

A **trace** is the full journey of a single request. It is made of **spans**, each representing one unit of work (a handler, a database query, an outgoing call) with a start time, duration and attributes. Spans form a tree: the root span is the incoming request and the children are the work it triggered.

```
GET /checkout                          6.41 s
  auth.verify                          0.02 s
  inventory.reserve   (HTTP call)      6.20 s   <- the problem
```

```
    db.query  SELECT ... FROM stock        6.15 s
  payment.charge                           0.15 s
```

This single picture would take an hour to reconstruct from logs. To work, services must pass **trace context** to each other, usually in the standard `traceparent` HTTP header, so spans from different services join into one trace. OpenTelemetry libraries do this for common frameworks automatically.

Connect the signals: include the **trace ID** in every log line, and the log viewer can jump straight to the trace, and the trace to the logs.

Sampling and cost

Storing every trace and every debug log is expensive and rarely useful.

Technique	Idea
Head sampling	Decide at the start of a request to keep, say, 5%
Tail sampling	Decide after the request ends; keep all traces with errors or high latency
Log level discipline	debug off in production; turn on briefly for one service when needed
Retention tiers	Keep recent logs hot and searchable, older logs in cheap storage, delete after the required period

Tail sampling is the most useful for diagnosis: you keep exactly the slow and failing requests, which are the ones you will want to look at.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Logs and distributed tracing fits together

Part	What it does	What goes wrong
Structured logs	Plain text logs are written for people and are hard for machines to search.	Unstructured text logs that cannot be queried.
Searching logs	Whatever the backend (Loki, Elasticsearch, a cloud log service), the workflow is the same: narrow by time, then service, then a field, then follow an ID.	No request or trace ID in log lines.
Distributed tracing	A trace is the full journey of a single request.	Keeping every trace and debug log forever.
Sampling and cost	Storing every trace and every debug log is expensive and rarely useful.	A custom HTTP client that drops trace headers.

Flow: Structured logs to Searching logs / Searching logs to Distributed tracing / Distributed tracing to Sampling and cost

Incident: a trace is missing half its services

> **Incident:** *Traces for checkout show the API and the payment service, but the inventory service never appears, even though logs prove it handled the request.*

The trace was broken at a boundary: a hop did not carry the trace context forward.

```
$ curl -sI -H 'traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01' http://invent \
    ory/health
$ kubectl logs deploy/inventory -n shop --tail 2
{"msg":"request","request_id":"c7d1e2","traceparent":null}
```

`traceparent` is `null` on arrival, so inventory started a new trace. The API called it with a hand-written HTTP client that did not copy the headers. Use the instrumented client (or inject the context explicitly), redeploy and view a fresh trace to confirm inventory's spans now appear as children. A good test to keep: after any change to service-to-service calls, check that a test request produces one trace spanning all services.

MARGA by EknathaLabs
marga.eknathalabs.com

Common mistakes

- Unstructured text logs that cannot be queried
- Logging sensitive data
- No request or trace ID in log lines
- Keeping every trace and debug log forever
- A custom HTTP client that drops trace headers

Check yourself

Why prefer structured logs?

Answer: Fields can be filtered and aggregated reliably

What is a span?

Answer: One unit of work within a trace, with a start time and duration

What does tail sampling let you do?

Answer: Decide after the request finishes and keep the slow or failing traces

Where should containerised applications write logs?

Answer: To standard output, collected by the platform

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Log pipelines with Loki, Elasticsearch and Fluent Bit

15 minutes. Collect, parse, enrich and store logs at scale, choose between Loki and Elasticsearch, query with LogQL, and control volume, retention and cost.

The earlier lesson explained why structured logs matter. This one covers how logs travel from a container to a place where you can search them at 3 a.m., and how to keep that pipeline from becoming your largest bill.

What you will learn

- The log pipeline: collect, process, store, query
- Fluent Bit and the Kubernetes log layout
- Loki versus Elasticsearch, and what each is good at
- LogQL queries, retention and cost control

The pipeline

1. **Produce**: applications write to standard output as JSON
2. **Collect**: an agent on each node reads the log files
3. **Process**: parse, add labels (namespace, Pod, app), filter and redact
4. **Store**: a log backend indexes and keeps the data
5. **Query and alert**: people search, dashboards chart counts, rules alert

In Kubernetes, the container runtime writes each container's output to a file under `/var/log/pods/` on its node. A **DaemonSet** log agent reads these files, so applications need no logging library or agent of their own.

```
$ kubectl logs deploy/shop-api -n shop --tail=5
$ kubectl exec -n logging ds/fluent-bit -- ls /var/log/containers | head -3
$ kubectl get ds -n logging
```

Collecting with Fluent Bit

Fluent Bit is a lightweight agent that reads, parses and forwards logs.

```
[SERVICE]
    Flush        5
    Log_Level    info
    Parsers_File parsers.conf

[INPUT]
    Name         tail
    Path         /var/log/containers/*.log
    Parser       cri
    Tag          kube.*
    Mem_Buf_Limit 20MB

[FILTER]
    Name         kubernetes
    Match        kube.*
    Merge_Log    On
    Keep_Log     Off

[FILTER]
    Name         grep
    Match        kube.*
    Exclude      log healthz

[OUTPUT]
    Name         loki
    Match        kube.*
```

MARGA by EknathaLabs marga.eknathalabs.com

```
    Host         loki.logging.svc
    Port         3100
    Labels       job=fluentbit, namespace=$kubernetes['namespace_name'], app=$kubernetes['labels'] \
    ['app']
```

`Mem_Buf_Limit` stops a slow backend from exhausting the agent's memory. The `kubernetes` filter adds Pod metadata. The `grep` filter drops noisy health-check lines **before** they cost money. Always redact secrets and personal data at this stage if applications might log them.

Loki or Elasticsearch

Question	Loki	Elasticsearch / OpenSearch
What is indexed	Only labels; log text is stored compressed	Every word, in an inverted index
Storage cost	Low	High
Query speed for free text	Slower, scans matching streams	Fast full-text search
Operations	Simple, uses object storage	Heavier: shards, heap, upgrades
Fits with	Grafana, Prometheus-style labels	Complex search, analytics, security use cases

Loki follows the Prometheus idea: label by a few low-cardinality dimensions (`namespace`, `app`, `env`) and search within them. **Never** use high-cardinality values such as request IDs or user IDs as labels, because each combination becomes a separate stream and the index explodes. Elasticsearch costs more to run but suits teams who need rich free-text search and aggregations across large volumes.

Querying with LogQL

```
{namespace="shop", app="shop-api"} |= "error"
{namespace="shop", app="shop-api"} | json | level="error" | status >= 500
sum by (app) (rate({namespace="shop"} |= "error" [5m]))
topk(5, sum by (route) (count_over_time({app="shop-api"} | json | status >= 500 [1h])))
```

The first part selects streams by label, the pipeline then filters and parses lines, and the metric forms turn logs into numbers you can graph or alert on. Always select by labels first so the system reads less data.

```
$ logcli query '{namespace="shop"} |= "timeout"' --since=30m --limit=20
$ logcli labels namespace
```

Retention, volume and cost

- Set retention per environment and per importance, for example 7 days for debug, 30 for application logs, a year for audit
- Drop or sample debug logs in production
- Keep a **log volume dashboard** by namespace and alert on sudden growth
- Send long-term copies to object storage with lifecycle rules
- Use rate limits per tenant so a noisy service cannot take down the log system

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Log pipelines with Loki, Elasticsearch and Fluent... fits together

Part	What it does	What goes wrong
The pipeline	Produce: applications write to standard output as JSON.	Logging secrets, tokens or personal data.
Collecting with Fluent Bit	Fluent Bit is a lightweight agent that reads, parses and forwards logs.	Using high-cardinality labels in Loki.
Loki or Elasticsearch	Loki follows the Prometheus idea: label by a few low-cardinality dimensions (namespace, app, env) and search within them.	Shipping every debug line from production.
Querying with LogQL	The first part selects streams by label, the pipeline then filters and parses lines, and the metric forms turn logs into numbers you can graph or alert on.	No parsing, so everything is one unsearchable text blob.
Retention, volume and cost	Set retention per environment and per importance, for example 7 days for debug, 30 for application logs, a year for audit.	No monitoring of the logging system itself.

Flow: The pipeline to Collecting with Fluent Bit / Collecting with Fluent Bit to Loki or Elasticsearch / Loki or Elasticsearch to Querying with LogQL / Querying with LogQL to Retention, volume and cost

Incident: logs stop when the disk fills

> **Incident:** *An outage investigation begins, and the search backend returns nothing newer than two hours ago. The logging stack's disk filled during a spike when a service started printing stack traces in a tight loop, then the indexer stopped.*

Check each stage, starting from the end:

```
$ kubectl get pods -n logging
$ kubectl logs sts/loki -n logging --tail=20 | grep -iE "error|no space"
$ kubectl exec -n logging sts/loki -- df -h /data
$ kubectl top pods -n shop --sort-by=memory | head -3
```

Free space (delete the oldest data or expand the volume), restart the writer, and find the noisy source. For the future: enforce per-tenant ingestion limits, alert when log storage passes 70 percent, add retention, and keep a fallback of `kubectl logs` and node files for the moments when the log system itself is down.

Common mistakes

- Using high-cardinality labels in Loki
- Shipping every debug line from production
- No parsing, so everything is one unsearchable text blob
- No monitoring of the logging system itself
- Logging secrets, tokens or personal data

Check yourself

What does Loki index?

Answer: Only the labels attached to log streams

Why avoid putting a request ID in a Loki label?

Answer: Each unique value creates a new stream and causes cardinality problems

Why run the log agent as a DaemonSet?

Answer: One agent per node can read all container log files on that node

Where is the best place to drop noisy health-check logs?

Answer: In the collection pipeline, before they are stored

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 5. Instrumenting with OpenTelemetry

16 minutes. Emit traces, metrics and logs with a vendor-neutral standard, add auto and manual instrumentation, route data through the Collector and keep context across services.

Every monitoring vendor used to ship its own agents and libraries, and switching meant re-instrumenting every service. **OpenTelemetry** (OTel) is the open standard that fixed this. You instrument once, then send the data to any compatible backend: Prometheus, Jaeger, Tempo, Grafana, Datadog and many others.

What you will learn

- The OTel building blocks: API, SDK, exporters and the Collector
- Automatic and manual instrumentation
- Context propagation across services
- Collector pipelines, sampling and resource attributes

Building blocks

- **API and SDK**: libraries in your language. The API is what code calls, and the SDK implements it
- **Signals**: traces, metrics and logs share common naming and context
- **Exporter**: sends data out, usually using the OTLP protocol
- **Collector**: a separate service that receives, processes and forwards telemetry
- **Semantic conventions**: standard attribute names such as `http.request.method` and `service.name`

Sending data to a Collector rather than directly to a vendor lets you change backends, add filtering and control cost without redeploying applications.

Automatic instrumentation

For common frameworks, auto-instrumentation wraps HTTP servers, clients and database drivers with no code change.

```
$ python -m pip install opentelemetry-distro opentelemetry-exporter-otlp
$ opentelemetry-bootstrap -a install
$ export OTEL_SERVICE_NAME=shop-api
$ export OTEL_RESOURCE_ATTRIBUTES=deployment.environment=prod,service.version=1.8.2
$ export OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector.observability:4317
$ opentelemetry-instrument python main.py
```

Within minutes you get a trace for each request with spans for the HTTP handler, outgoing calls and database queries. Always set `service.name`, `service.version` and `deployment.environment`: they are how you filter and compare across releases.

Manual instrumentation

Add spans around business operations that frameworks cannot know about.

```
from opentelemetry import trace

tracer = trace.get_tracer("shop.checkout")

def place_order(cart):
    with tracer.start_as_current_span("place_order") as span:
        span.set_attribute("cart.items", len(cart.items))
        span.set_attribute("customer.tier", cart.customer.tier)
        try:
            charge(cart)
        except PaymentError as exc:
            span.record_exception(exc)
            span.set_status(trace.Status(trace.StatusCode.ERROR))
            raise
        reserve_stock(cart)
```

MARGA by EknathaLabs marga.eknathalabs.com

Attributes should help you answer questions during an incident. Avoid putting personal data or secrets in them, and avoid unbounded values (full URLs with IDs) as metric dimensions.

Context propagation

For a trace to cross services, each service passes the trace context to the next, normally in the W3C `traceparent` header.

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
             ver  trace-id (32 hex)               parent span (16 hex) flags
```

HTTP clients and servers handle this automatically once instrumented. It breaks at queues, batch jobs and custom protocols unless you inject and extract the context yourself. A trace that stops at a message broker is the most common gap.

The Collector

```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

processors:
  memory_limiter:
    check_interval: 1s
    limit_mib: 400
  batch:
    timeout: 5s
  attributes/redact:
    actions:
      - key: http.request.header.authorization
        action: delete
  probabilistic_sampler:
    sampling_percentage: 10

exporters:
  otlp/tempo:
    endpoint: tempo.observability:4317
    tls:
      insecure: true
  prometheusremotewrite:
    endpoint: http://mimir.observability/api/v1/push

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, attributes/redact, probabilistic_sampler, batch]
      exporters: [otlp/tempo]
    metrics:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [prometheusremotewrite]
```

A pipeline connects receivers, processors and exporters. `memory_limiter` protects the Collector, `batch` improves efficiency and the redaction processor removes sensitive attributes. Run it as an agent on each node and as a gateway deployment for central processing.

Sampling

Storing every trace is expensive. **Head sampling** decides when the trace starts (simple, but may drop the interesting ones). **Tail sampling** decides after the trace completes, so you can keep every trace with an error or high latency and sample the rest.

```
$ kubectl logs deploy/otel-collector -n observability --tail=20
$ curl -s localhost:8888/metrics | grep -E "otelcol_(receiver_accepted|exporter_sent|exporter_send \
    _failed)"
```

MARGA by EknathaLabs marga.eknathalabs.com

Monitor the Collector itself: dropped or failed exports mean you have blind spots.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Instrumenting with OpenTelemetry fits together

Part	What it does	What goes wrong
Building blocks	API and SDK: libraries in your language.	Sending directly to a vendor from every application.
Automatic instrumentation	For common frameworks, auto-instrumentation wraps HTTP servers, clients and database drivers with no code change.	Skipping service.name, so everything appears as "unknown_service".
Manual instrumentation	Add spans around business operations that frameworks cannot know about.	Putting personal data into span attributes.
Context propagation	For a trace to cross services, each service passes the trace context to the next, normally in the W3C traceparent header.	It breaks at queues, batch jobs and custom protocols unless you inject and extract the context yourself.
The Collector	A pipeline connects receivers, processors and exporters.	Sampling away error traces.
Sampling	Storing every trace is expensive. Head sampling decides when the trace starts (simple, but may drop the interesting ones).	Not monitoring the Collector.

Flow: Building blocks to Automatic instrumentation / Automatic instrumentation to Manual instrumentation / Manual instrumentation to Context propagation / Context propagation to The Collector / The Collector to Sampling

Incident: traces stop at the queue

> **Incident:** *Slow checkouts are reported. The trace for a slow order shows the API call taking 80 ms, then ends. The delay is somewhere afterwards, in a worker that processes the payment queue, but the trace has no spans from it.*

Context was lost at the message queue. The producer must inject the context into the message and the consumer must extract it.

```
from opentelemetry import trace, propagate

def publish(queue, payload):
    headers = {}
    propagate.inject(headers)
    queue.send(payload, attributes=headers)

def handle(message):
    ctx = propagate.extract(message.attributes)
    tracer = trace.get_tracer("shop.worker")
    with tracer.start_as_current_span("process_payment", context=ctx):
        charge_card(message.body)
```

With propagation in place the worker's spans join the original trace, and the slow step (a 6 second call to the payment provider) is visible immediately. Add this pattern to your service template so every new queue consumer does it.

Common mistakes

- Skipping `service.name`, so everything appears as "unknown_service"
- Sending directly to a vendor from every application
- Sampling away error traces
- Putting personal data into span attributes
- Not monitoring the Collector

MARGA by EknathaLabs
marga.eknathalabs.com

Check yourself

What problem does OpenTelemetry solve?

Answer: It provides a vendor-neutral standard for instrumenting and exporting telemetry

What does the Collector add?

Answer: A central place to receive, process, filter and route telemetry

How does a trace cross from one service to another?

Answer: The trace context is propagated, normally in the traceparent header

Why use tail sampling?

Answer: Decisions can keep all error and slow traces after seeing the whole trace

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Blackbox probes and synthetic monitoring

14 minutes. Measure availability from the user's side with the Prometheus blackbox exporter and synthetic checks, probe HTTP, TCP, DNS and certificates, and alert on what users actually experience.

Whitebox monitoring reads metrics that the system reports about itself. If the load balancer, DNS or certificate is broken, the application may report that everything is fine because no requests reach it. **Blackbox monitoring** tests from the outside, the way a user would, and catches failures that internal metrics cannot see.

What you will learn

- Whitebox versus blackbox monitoring
- Running the Prometheus blackbox exporter with HTTP, TCP, DNS and ICMP probes
- Certificate expiry monitoring
- Multi-step synthetic checks and where to run probes from

Two viewpoints

	Whitebox	Blackbox
Source	Metrics exposed by the app and its host	A probe acting as a client
Finds	Saturation, internal errors, slow queries	Outages, DNS and TLS problems, routing, CDN faults
Misses	Failures before the request reaches the app	Why something failed internally
Best for	Diagnosis	Detection and availability SLIs

You need both. Blackbox tells you something is wrong right now; whitebox tells you why.

The blackbox exporter

The exporter performs the probe on request. Prometheus passes the target as a parameter and scrapes the result.

```
modules:
  http_2xx:
    prober: http
    timeout: 5s
    http:
      method: GET
      valid_status_codes: [200]
      fail_if_not_ssl: true
      preferred_ip_protocol: ip4
  http_json_ok:
    prober: http
    timeout: 5s
    http:
      fail_if_body_not_matches_regexp:
        - '"status":\s*"ok"'
  tcp_connect:
    prober: tcp
    timeout: 3s
  dns_a:
    prober: dns
    timeout: 3s
    dns:
      query_name: shop.example.com
      query_type: A

scrape_configs:
  - job_name: blackbox-http
```

MARGA by EknathaLabs marga.eknathalabs.com

```
    metrics_path: /probe
    params:
      module: [http_2xx]
    static_configs:
      - targets:
          - https://shop.example.com/healthz
          - https://api.example.com/v1/status
    relabel_configs:
      - source_labels: [__address__]
        target_label: __param_target
      - source_labels: [__param_target]
        target_label: instance
      - target_label: __address__
        replacement: blackbox-exporter.observability:9115
```

Try a probe by hand to see the output:

```
$ curl -s 'http://localhost:9115/probe?target=https://shop.example.com/healthz&module=http_2xx' |  \
    grep -E "^probe_(success|http_status_code|duration_seconds|ssl_earliest_cert_expiry)"
probe_success 1
probe_http_status_code 200
probe_duration_seconds 0.184
probe_ssl_earliest_cert_expiry 1.7615e+09
```

Useful alerts

```
# the endpoint is failing from the outside
probe_success{job="blackbox-http"} == 0

# slow responses
probe_duration_seconds{job="blackbox-http"} > 1.5

# certificate expires within 21 days
(probe_ssl_earliest_cert_expiry - time()) / 86400 < 21

# availability over 30 days, a ready-made SLI
avg_over_time(probe_success{instance="https://shop.example.com/healthz"}[30d])
```

Use `for: 2m` so one failed probe does not page anyone. The certificate alert catches expiring certificates for any host, including those managed by systems you do not control. It would have prevented a large share of all famous outages.

Synthetic journeys

A single GET shows the site responds. A **synthetic transaction** drives a real flow: log in, search, add to cart, check out in a test account. Tools such as Grafana k6, Playwright-based monitors and cloud services (CloudWatch Synthetics, Application Insights availability tests) run them on a schedule.

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = { vus: 1, duration: '30s', thresholds: { checks: ['rate>0.99'] } };

export default function () {
  const r = http.get('https://shop.example.com/api/products?limit=1');
  check(r, { 'status is 200': (res) => res.status === 200, 'has items': (res) => res.json('items') \
    .length > 0 });
  sleep(5);
}
```

Use dedicated test accounts and mark the traffic so analytics and billing ignore it. Keep the journey short and stable, because a flaky synthetic check teaches the team to ignore alerts.

Where to probe from

A probe running inside the same cluster as the service cannot see an ingress, DNS or internet-path failure. Run probes from **outside** the failure domain: a second region, another cloud, or a hosted probing service with many locations. Alert when a majority of locations fail, to avoid paging for a problem on the probe's own network.

MARGA by EknathaLabs
marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Blackbox probes and synthetic monitoring fits together

Part	What it does	What goes wrong
Two viewpoints	You need both. Blackbox tells you something is wrong right now; whitebox tells you why.	Paging on a single failed probe.
The blackbox exporter	The exporter performs the probe on request.	Forgetting certificate expiry alerts.
Useful alerts	Use for: 2m so one failed probe does not page anyone.	Probing only an internal health endpoint.
Synthetic journeys	A single GET shows the site responds. A synthetic transaction drives a real flow: log in, search, add to cart, check out in a test account.	Synthetic tests that use real customer data or are flaky.
Where to probe from	A probe running inside the same cluster as the service cannot see an ingress, DNS or internet-path failure.	Probes running in the same failure domain as the service.

Flow: Two viewpoints to The blackbox exporter / The blackbox exporter to Useful alerts / Useful alerts to Synthetic journeys / Synthetic journeys to Where to probe from

Incident: green dashboards, dead website

> **Incident:** *At 14:05 a certificate renewal job fails silently and the certificate expires. Browsers show a full-page warning. Internal dashboards stay green because the application and the load balancer health checks use an internal plain-HTTP endpoint.*

Customers could not reach the site for 50 minutes. The blackbox probe would have flagged it in two:

```
$ echo | openssl s_client -connect shop.example.com:443 -servername shop.example.com 2>/dev/null | \
     openssl x509 -noout -enddate
notAfter=Jun  2 14:00:00 2025 GMT
$ curl -sS -m 5 -o /dev/null -w "%{http_code}\n" https://shop.example.com/
curl: (60) SSL certificate problem: certificate has expired
```

Renew and deploy the certificate, then add the `probe_ssl_earliest_cert_expiry` alert, an external HTTPS probe on the public hostname and a warning 21 days before expiry.

Common mistakes

- Probing only an internal health endpoint
- Probes running in the same failure domain as the service
- Paging on a single failed probe
- Synthetic tests that use real customer data or are flaky
- Forgetting certificate expiry alerts

Check yourself

What can blackbox monitoring detect that whitebox often cannot?

Answer: DNS, TLS and routing failures that stop requests reaching the application

Which metric shows whether a probe succeeded?

Answer: probe_success

Why run probes from outside the cluster or region?

Answer: Probes inside the failure domain cannot see failures in the path users take

How early should a certificate-expiry alert fire?

MARGA by EknathaLabs
marga.eknathalabs.com

Answer: Weeks ahead, for example 21 days, so renewal problems are fixed calmly

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Alerting and dashboards

16 minutes. Alert on symptoms that matter to users, write alert rules with Alertmanager routing, and design dashboards that answer questions quickly.

A pager that rings at 3 a.m. for a problem nobody needs to fix trains people to ignore it. An alert that never fires during a real outage costs more. Good alerting is a product design problem: every alert must be urgent, actionable and clear about what is wrong.

What you will learn

- What deserves a page versus a ticket
- Writing and routing Prometheus alerts
- Multi-window burn-rate alerts
- Dashboard design

What deserves an alert

Ask three questions of every proposed alert:

1. **Does a human need to act, right now?** If not, it is a ticket or a dashboard, not a page.
2. **Is it a symptom users feel?** Prefer "checkout error ratio above 2%" over "CPU above 80%". Causes can be investigated after a symptom fires.
3. **Does the responder know what to do?** Every page links to a runbook.

Kind	Delivered as	Example
Urgent, user-impacting	Page	Error ratio high, SLO burning fast
Needs action soon	Ticket or chat message	Disk will fill in 3 days, certificate expires in 14 days
Informational	Dashboard	CPU trends, deployment markers

Alerts on causes still have a place when they predict a symptom, such as "disk will fill in 4 hours", because they give you time to act before users are hurt.

Writing alert rules

```
groups:
  - name: shop-api.alerts
    rules:
      - alert: ShopApiHighErrorRatio
        expr: |
          sum(rate(http_requests_total{service="shop-api", status=~"5.."}[5m]))
            /
          sum(rate(http_requests_total{service="shop-api"}[5m])) > 0.02
        for: 10m
        labels:
          severity: page
          team: shop
        annotations:
          summary: "Shop API error ratio above 2% for 10 minutes"
          runbook: "https://runbooks.example.com/shop-api/high-errors"

      - alert: InstanceDown
        expr: up{job="shop-api"} == 0
        for: 5m
        labels:
          severity: ticket
        annotations:
          summary: "{{ $labels.instance }} has stopped reporting"
```

The `for:` clause requires the condition to hold continuously, which filters brief spikes. Labels drive routing, and annotations explain the alert to the responder.

MARGA by EknathaLabs marga.eknathalabs.com

Routing with Alertmanager

Prometheus evaluates rules; **Alertmanager** deduplicates, groups, silences and routes the resulting alerts.

```
route:
  receiver: team-chat
  group_by: [alertname, service]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  routes:
    - matchers: ['severity="page"']
      receiver: on-call-pager

receivers:
  - name: on-call-pager
    pagerduty_configs:
      - routing_key_file: /etc/alertmanager/secrets/pager-key
  - name: team-chat
    slack_configs:
      - api_url_file: /etc/alertmanager/secrets/slack-url
        channel: "#shop-alerts"
```

Grouping means fifty failing instances produce one notification instead of fifty. **Inhibition rules** can mute downstream alerts when a root cause alert is already firing (for example, mute service alerts when the whole cluster is unreachable).

Alert on the budget, not the blip

With service level objectives (covered in the SRE module), the best alerts measure how fast you are consuming your **error budget**. A **burn rate** of 1 uses the budget exactly over the whole window; a burn rate of 14 would exhaust a 30-day budget in about two days. A common design pairs a short and a long window, so you are paged only for sustained, serious burning and not for a blip:

```
      - alert: ShopApiBudgetBurnFast
        expr: |
          (
            sum(rate(http_requests_total{service="shop-api",status=~"5.."}[5m]))
              / sum(rate(http_requests_total{service="shop-api"}[5m]))
          ) > (14 * 0.001)
          and
          (
            sum(rate(http_requests_total{service="shop-api",status=~"5.."}[1h]))
              / sum(rate(http_requests_total{service="shop-api"}[1h]))
          ) > (14 * 0.001)
        for: 2m
        labels:
          severity: page
```

Here 0.001 is the error budget for a 99.9% objective. The long window proves it is not momentary and the short window makes the alert resolve quickly when the problem ends.

Dashboards

A dashboard should answer a question in under ten seconds.

- Put the **user-facing signals first**: request rate, error ratio, latency percentiles. Resource graphs go below.
- One dashboard per service with a fixed layout, so people know where to look at 3 a.m.
- Show **deployment and incident markers** on graphs; many problems start at a release.
- Use consistent units and time ranges, and label axes.
- Link from an alert to the dashboard panel that shows the same query.
- Prefer a few meaningful panels to forty. Delete panels nobody looks at.

MARGA by EknathaLabs
marga.eknathalabs.com

Incident: alert fatigue

> **Incident:** *The on-call engineer received 214 pages last week. Most were closed without action, and a real database alert was acknowledged and then forgotten.*

Treat the alert set as something that is itself in need of maintenance. Pull the numbers:

```
$ amtool alert query --active -o json | jq -r '.[].labels.alertname' | sort | uniq -c | sort -rn | \
    head
```

Three alerts account for 80% of the volume: a CPU alert with no impact, a flapping disk check, and a duplicate of the error-ratio alert. For each ask: was action required? If no, remove it, convert it to a ticket, or lengthen its `for:` period. Add a weekly review of every page: who was paged, was it actionable, what changed. Track pages per shift as a team metric. A healthy target is only a handful per shift, and each one meaningful.

Diagram: From a signal to a person

Part	What it does	What goes wrong
Metric	Prometheus evaluates a rule on the data.	Wrong labels make the alert silent.
Alert rule	Fires after the condition holds for a duration.	A zero duration pages on every blip.
Alertmanager	Groups, deduplicates and routes alerts.	Wrong routing sends pages to nobody.
Pager or chat	The on-call person receives it with a runbook link.	Without a runbook the person starts from scratch.
Dashboard	Shows context for investigation.	Too many panels hide the one that matters.

Flow: Metric to Alert rule / Alert rule to Alertmanager / Alertmanager to Pager or chat / Alert rule to Dashboard

Common mistakes

- Paging on causes with no user impact
- Alerts with no runbook
- No `for:` duration, so every spike pages
- Dashboards with no owner and no clear question
- Never reviewing alert quality

Check yourself

Which is a better page-worthy alert?

Answer: Checkout error ratio above 2% for 10 minutes

What does the for: clause in an alert rule do?

Answer: Requires the condition to hold continuously for that duration before firing

What is Alertmanager's job?

Answer: Deduplicating, grouping, silencing and routing alerts

What does a burn rate of 1 mean?

Answer: The error budget is being consumed at exactly the pace that uses it up over the full window

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. RED, USE and the golden signals in practice

13 minutes. Pick the right small set of metrics for every service and resource using the RED method, the USE method and the four golden signals, and build dashboards that answer questions fast.

A dashboard with eighty graphs tells you nothing during an incident. Experienced teams start with a short, standard set of signals for every service, so that anyone can open any dashboard and know where to look. Three frameworks give you that set. They look similar, but each fits a different kind of thing.

What you will learn

- The RED method for request-driven services
- The USE method for resources
- Google's four golden signals
- Writing the queries and choosing percentiles
- Dashboard layout for fast diagnosis

RED: for services

For every service that handles requests, track:

- **Rate:** requests per second
- **Errors:** failed requests per second, or the error ratio
- **Duration:** how long requests take, as percentiles (p50, p95, p99), never only the average

```
sum(rate(http_requests_total{service="checkout"}[5m]))
sum(rate(http_requests_total{service="checkout",status=~"5.."}[5m]))
  / sum(rate(http_requests_total{service="checkout"}[5m]))
histogram_quantile(0.99, sum by (le) (rate(http_request_duration_seconds_bucket{service="checkout" \
    }[5m])))
```

USE: for resources

For every resource (CPU, memory, disks, network links, connection pools, queues), track:

- **Utilisation:** the share of time or capacity in use
- **Saturation:** extra work waiting (run queue length, swap, queue depth)
- **Errors:** error counts (disk errors, dropped packets)

Saturation is the signal people forget: a disk may be 60 percent utilised and yet have a long queue; a thread pool may be fine on average yet have requests waiting for a free thread.

The four golden signals

The SRE book's version combines both: **latency** (separate successful from failed requests, since fast errors hide slow problems), **traffic**, **errors**, and **saturation**. If you can only monitor four things about a user-facing system, monitor these.

Framework	Applies to	Core question
RED	Services	Are users getting fast, correct answers?
USE	Resources	Is something running out?
Golden signals	Whole systems	Is the system healthy and how close to its limit?

MARGA by EknathaLabs marga.eknathalabs.com

Percentiles, not averages

If 1 percent of requests take ten seconds, the average barely moves but one user in a hundred has a bad experience, and with many requests per page, most users will meet a slow one. Use histograms to compute percentiles, and be careful: you cannot average percentiles across instances. Aggregate the histogram buckets first, then take the quantile.

A dashboard that works

Put the user-facing view at the top: RED for the service, with the objective line drawn. Below it, the dependencies (RED for each downstream service), then resources (USE) for the hosts or Pods. Add deploy markers and annotations so changes line up with the graphs. Link each panel to the underlying query, logs and traces. The same layout for every service means an on-call person who has never seen this service can still find the problem.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How RED, USE and the golden signals in practice fits together

Part	What it does	What goes wrong
RED: for services	For every service that handles requests, track.	Only utilisation, no saturation.
USE: for resources	For every resource (CPU, memory, disks, network links, connection pools, queues), track.	Check the Common mistakes list below this diagram for what usually goes wrong here.
The four golden signals	The SRE book's version combines both: latency (separate successful from failed requests, since fast errors hide slow problems), traffic, errors, and saturation.	Averages for latency.
Percentiles, not averages	If 1 percent of requests take ten seconds, the average barely moves but one user in a hundred has a bad experience, and with many requests per page, most users will meet...	Averaging percentiles across instances.
A dashboard that works	Put the user-facing view at the top: RED for the service, with the objective line drawn.	Different dashboard layouts for each service.

Flow: RED: for services to USE: for resources / USE: for resources to The four golden signals / The four golden signals to Percentiles, not averages / Percentiles, not averages to A dashboard that works

Incident: the green dashboard and the slow checkout

> **Incident:** *Customers complain about slow checkout while the dashboard shows CPU at 35 percent and average latency of 220 ms.*

```
histogram_quantile(0.99, sum by (le) (rate(http_request_duration_seconds_bucket{service="checkout" \
    }[5m])))
```

The p99 was 9 seconds. The service's database connection pool was saturated: requests waited for a free connection, but CPU stayed low and the average hid the tail. The team added p95/p99 panels, a pool saturation panel (in-use versus size and wait time), and an alert on p99 against the SLO.

Common mistakes

- Averages for latency
- Only utilisation, no saturation
- Averaging percentiles across instances
- Different dashboard layouts for each service

MARGA by EknathaLabs
marga.eknathalabs.com

Check yourself

What do the letters in RED stand for?

Answer: Rate, Errors, Duration

Which signal does the USE method add that people often forget?

Answer: Saturation

Why use percentiles for latency?

Answer: Averages hide the slow tail that real users feel

How do you compute a fleet-wide p99 correctly?

Answer: Sum the histogram buckets across instances, then compute the quantile

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Telemetry cost, cardinality and retention

13 minutes. Keep monitoring affordable and fast by controlling metric cardinality, log volume, trace sampling and retention, and by tracking the cost of observability as a budget.

Observability data grows faster than the systems it describes. Without control, the monitoring bill rivals the infrastructure bill, dashboards time out, and teams start turning things off, usually the wrong things. Good teams treat telemetry as a product with a budget, owners and design rules.

What you will learn

- Why metric cardinality is the main cost driver
- Finding and fixing high-cardinality labels
- Log volume control: levels, sampling, drop rules and tiering
- Trace sampling strategies
- Retention, downsampling and cost attribution

Cardinality

Each unique combination of label values creates a separate time series. A metric with labels `service` (50 values), `endpoint` (40), `status` (10) and `region` (5) can create 100,000 series. Add a `user_id` label with a million values and you create billions. Memory, query time and cost grow with series count, not with samples.

Rules:

- Allowed labels have a small, bounded set of values: service, region, status class, method.
- Never put unbounded values in labels: user ids, request ids, email addresses, full URLs with ids, timestamps.
- Normalise paths: `/users/{id}`, not `/users/12345`.
- Put unique details in logs and traces, which are designed for them.

```
topk(10, count by (__name__) ({__name__=~".+"}))          # which metrics have the most series
count(count by (endpoint) (http_requests_total))          # how many endpoint values exist
```

Most systems also provide a cardinality explorer, and a way to limit series per target at scrape time.

Logs

Logs are usually the largest volume. Ways to control them:

- Keep the default level at info; debug only temporarily, per service.
- Do not log inside tight loops or full request bodies.
- Drop known noise (health check requests) in the collector.
- Sample repetitive messages, keeping the first and a count.
- Tier storage: recent logs on fast storage for a week or two, older logs on cheap object storage, searchable on demand.
- Mask secrets and personal data before storage; this is a security rule and a cost rule.

MARGA by EknathaLabs marga.eknathalabs.com

Traces

Tracing every request in a busy system is expensive. **Head sampling** decides at the start (for example keep 5 percent) and is cheap but may miss the rare slow or failed request. **Tail sampling** decides after the trace is complete in a collector, keeping all errors and all slow traces and a small sample of the rest, but it needs the collector to hold traces in memory for a time.

```
processors:
  tail_sampling:
    decision_wait: 10s
    policies:
      - { name: errors,  type: status_code, status_code: { status_codes: [ERROR] } }
      - { name: slow,    type: latency,     latency: { threshold_ms: 1000 } }
      - { name: sample,  type: probabilistic, probabilistic: { sampling_percentage: 5 } }
```

Retention and downsampling

Keep raw metrics at full resolution for days or weeks, then downsample (for example to five-minute averages) for months, because old data is read for trends, not detail. Decide retention per data type from real needs: incident analysis, capacity planning, audits and legal rules.

Cost as a managed budget

Attribute volume and cost to teams (by label or by index), show them their usage, and set alerts on growth. Review the top ten series, log sources and trace producers each month. A sudden jump is often a deploy that added a bad label or a debug log.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Telemetry cost, cardinality and retention fits together

Part	What it does	What goes wrong
Cardinality	Each unique combination of label values creates a separate time series.	Unbounded label values.
Logs	Logs are usually the largest volume.	Keeping debug logs on forever.
Traces	Tracing every request in a busy system is expensive.	Head-sampling away the failures.
Retention and downsampling	Keep raw metrics at full resolution for days or weeks, then downsample (for example to five-minute averages) for months, because old data is read for trends, not detail.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Cost as a managed budget	Attribute volume and cost to teams (by label or by index), show them their usage, and set alerts on growth.	No owner for the telemetry bill.

Flow: Cardinality to Logs / Logs to Traces / Traces to Retention and downsampling / Retention and downsampling to Cost as a managed budget

Incident: the monitoring system that fell over

Incident: *Prometheus runs out of memory and restarts every few minutes, and alerts stop for two hours.*

```
$ curl -s localhost:9090/api/v1/status/tsdb | jq '.data.seriesCountByMetricName[0:2]'
[{"name":"http_requests_total","value":4180233},{"name":"cart_item_latency_bucket","value":912004} \
    ]
```

A release added `customer_id` as a label on the request metric. The team removed the label, dropped the series with relabel rules, set a per-target series limit, added an alert on series growth and put label review in code review.

Common mistakes

- Unbounded label values
- Keeping debug logs on forever
- Head-sampling away the failures
- No owner for the telemetry bill

Check yourself

What mainly determines the cost and memory use of metrics?

Answer: The number of unique time series, from label combinations

Which is a safe metric label?

Answer: status class such as 2xx, 4xx, 5xx

What does tail sampling let you do?

Answer: Keep all error and slow traces while sampling the rest

Why downsample old metrics?

Answer: Old data is used for trends, so lower resolution saves storage and query time

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Structured logging and log design

12 minutes. Write logs that are easy to search: structured fields, consistent names, correlation IDs, sensible levels and no secrets.

During an incident, you search for a customer ID and get nothing, because one service logs `customer=42`, another `cust_id: 42`, and a third writes a sentence. Logs are only useful if you can find the right lines quickly. Designing them takes an hour at the start and saves days later.

What you will learn

- Structured logs as JSON with stable field names
- Correlation and trace IDs that tie lines across services
- What to log, what not to log, and which level to use
- Cost control: sampling and retention

From sentences to fields

A log line is an event with fields. Compare:

```
2026-01-05 10:02:11 payment failed for customer 42 after 3 tries
```

```
{"ts":"2026-01-05T10:02:11Z","level":"error","service":"payments","event":"payment_failed","custom \
    er_id":42,"attempts":3,"request_id":"r-7f3a"}
```

The second can be filtered, counted and graphed without writing a pattern for each message. Use one name for one meaning across all services (`customer_id`, not three spellings), use consistent units (milliseconds, bytes) and ISO 8601 timestamps in UTC. Keep the human message short and put details in fields.

Correlation IDs

A user request passes through several services. Generate a **request ID** at the edge (or use the trace ID from tracing), pass it in a header to every service, and add it to every log line. Then one search shows the whole journey of one request.

```
$ grep r-7f3a gateway.log orders.log payments.log
```

In a log system, filter on the field. Make the ID part of your logging setup (for example a context variable that the logger reads), so developers do not need to pass it by hand.

What to log

Log decisions and boundaries: request received and finished (with duration and status), calls to other systems and their results, retries, state changes, and unexpected conditions. Use levels as a contract: `error` means someone may need to act, `warn` means unusual but handled, `info` means normal milestones, `debug` is off in production. Never log passwords, tokens, full card numbers or personal data that you do not need. Mask or hash identifiers when you can, and review logs when you add a new field.

Cost and retention

Logs are often the biggest observability cost. Keep recent logs (for example 14 days) searchable and move older ones to cheap storage with a longer retention for audit. Reduce volume at the source: sample repetitive debug lines, drop health check requests, and set rate limits on noisy loops. Track log volume per service and give owners a budget. A service that writes 5 GB a day of identical lines is a bug, not a feature.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Structured logging and log design fits together

Part	What it does	What goes wrong
From sentences to fields	A log line is an event with fields.	Free-text sentences with data buried inside.
Correlation IDs	A user request passes through several services.	Different names for the same field.
What to log	Log decisions and boundaries: request received and finished (with duration and status), calls to other systems and their results, retries, state changes, and unexpected...	Logging secrets or personal data.
Cost and retention	Logs are often the biggest observability cost.	Keeping all logs forever at full price.

Flow: From sentences to fields to Correlation IDs / Correlation IDs to What to log / What to log to Cost and retention

Incident: Searching three spellings at 3 a.m.

> **Incident:** *A customer reported double charges, and the on-call engineer needed more than an hour to assemble what happened.*

```
$ grep -c "cust=" payments.log orders.log gateway.log
payments.log:4120
orders.log:0
gateway.log:0
$ grep -c "customer_id" orders.log
8815
```

Each team had invented its own format. The payments service used `cust=`, orders used JSON with `customer_id`, and the gateway logged only paths. After the incident the company defined a short logging standard with required fields (timestamp, service, level, event, request_id), added a library that applies it, and made a check in review that rejects new free-text-only logs for request handling.

Common mistakes

- Free-text sentences with data buried inside
- Different names for the same field
- Logging secrets or personal data
- Keeping all logs forever at full price

Check yourself

What is the main benefit of structured logs?

Answer: Fields can be filtered, counted and graphed without custom parsing

What is a correlation ID?

Answer: An identifier passed through services so one request can be followed everywhere

Which level fits a handled but unusual condition?

Answer: warn

How do you reduce log cost without losing needed data?

Answer: Sample noisy lines, drop health checks and tier old logs to cheap storage

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 11. Dashboards and runbooks for on-call

12 minutes. Design the few dashboards and runbooks an on-call engineer actually needs, and keep them accurate.

The page fires at 03:00, and the engineer opens a dashboard with sixty graphs and a runbook last edited two years ago. Both are present and neither helps. Good on-call material is small, current and organized around the questions an engineer asks in the first five minutes.

What you will learn

- The questions an on-call dashboard must answer
- A layered design: overview, service and deep dive
- Runbooks that start from the alert and give concrete steps
- Keeping both current

Design from the first five minutes

When an alert fires, the engineer asks: Is it real? How bad? Since when? What changed? Where is the problem? The top-level dashboard should answer the first three at a glance: request rate, error rate and latency against the SLO, with deploy markers on the graphs. Put the most important numbers at the top, use consistent time ranges, and show thresholds so a graph says "bad" without anyone calculating.

Layers

Use three levels and link them:

1. **Overview** for the whole product: golden signals per service, one row each, red or green.
2. **Service dashboard**: rate, errors, duration, saturation (CPU, memory, connections, queue depth), and dependencies.
3. **Deep dive**: the panels used only in an investigation, such as per-endpoint latency or database locks.

Each alert links directly to the right service dashboard, with the time range and filters already set. Avoid giant dashboards with every metric; a panel that no one has looked at in a quarter should go.

Runbooks that work

A runbook is tied to one alert. It should open with what the alert means and how to confirm it, then give numbered steps with commands to copy, and say what is safe to do and when to escalate.

```
Alert: CheckoutErrorRateHigh
Meaning: more than 2% of checkout requests fail for 10 minutes.
Confirm: dashboard "checkout" -> error panel; run: kubectl -n shop get pods
Step 1: if a deploy happened in the last hour, roll back: kubectl rollout undo deploy/checkout
Step 2: if payments timeouts rise, enable fallback: set flag payments.fallback=true
Escalate: if not resolved in 20 minutes, page the payments team
Owner: checkout team, reviewed 2026-01-05
```

Write for a tired reader. Short sentences, exact commands, no assumed knowledge.

Keep them honest

Stale material is worse than none, because people trust it. Give each dashboard and runbook an owner and a review date. Update the runbook as part of every postmortem action. After an incident, ask the responders which panel they wished they had and which runbook step was wrong, and fix both within a week. Test runbooks in game days: someone who did not write it follows it step by step, and the gaps show up quickly.

MARGA by EknathaLabs
marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Dashboards and runbooks for on-call fits together

Part	What it does	What goes wrong
Design from the first five minutes	When an alert fires, the engineer asks: Is it real?	One huge dashboard for everything.
Layers	Use three levels and link them.	Runbooks with commands that no longer work.
Runbooks that work	A runbook is tied to one alert. It should open with what the alert means and how to confirm it, then give numbered steps with commands to copy, and say what is safe to...	Alerts without a runbook link.
Keep them honest	Stale material is worse than none, because people trust it.	No owner or review date.

Flow: Design from the first five minutes to Layers / Layers to Runbooks that work / Runbooks that work to Keep them honest

Incident: The 03:00 dashboard

> **Incident:** *An engineer needed 25 minutes to find that a rollout was the cause of an error spike, although the deploy was visible in the pipeline.*

```
$ grafana-cli dashboards list | wc -l
214
$ ./check-links.sh runbooks/
runbooks/checkout.md: dashboard https://.../d/old-checkout -> 404
```

The alert linked to a dashboard that had been deleted, and the new one had no deploy markers. The team trimmed the overview to one row per service, added deploy annotations from the pipeline, made each alert link to a service dashboard with the right filters, and added a link checker to the pipeline that fails when a runbook points to a missing page.

Common mistakes

- One huge dashboard for everything
- Alerts without a runbook link
- Runbooks with commands that no longer work
- No owner or review date

Check yourself

Which question should the top dashboard answer first?

Answer: Is the service healthy against its objectives right now?

What belongs at the start of a runbook?

Answer: What the alert means and how to confirm it

How do you keep runbooks current?

Answer: Give them owners and review dates, and update them after every incident

Why add deploy markers to graphs?

Answer: They show whether a change coincides with a problem

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Debugging with metrics, logs and traces

15 minutes. Use metrics to find that something is wrong, traces to find where, and logs to find why, and learn to debug the monitoring itself when data is missing.

Monitoring exists so you can find a cause faster. The three signals answer different questions: metrics tell you that something changed, traces show where time or errors accumulate, and logs explain what happened at that point. Used in this order they shorten an investigation from hours to minutes.

What you will learn

- A path from alert to cause using all three signals
- Slicing metrics by labels to find the affected part
- Following a trace and jumping to the matching logs
- Debugging missing data, gaps and high cardinality

From symptom to cause

1. **Metrics**: which service, endpoint or region is affected? When did it start? Compare with a deploy marker.
2. **Traces**: pick a slow or failed request and see which span takes the time or returns the error.
3. **Logs**: open the logs for that trace id and read the message at the failing span.

```
sum by (route) (rate(http_requests_total{status=~"5.."}[5m]))
histogram_quantile(0.99, sum by (le, route) (rate(http_request_duration_seconds_bucket[5m])))
sum by (version) (rate(http_requests_total{status=~"5.."}[5m]))
```

The last query splits errors by version and often shows that only the new release is failing.

Link signals with a trace id

```
{"ts":"2026-10-12T09:14:03Z","level":"error","trace_id":"4bf92f35","route":"/checkout","msg":"paym \
    ent timeout","upstream":"payments","ms":5003}
```

A trace id in every log line lets you jump from a slow span to the exact log lines. Without it you search by time and guess.

When the data itself is missing

Symptom	Likely cause	Check
Gap in a graph	Scrape failed or the target restarted	up metric, target page
Flat line at zero	Counter reset or the label changed	Rename or relabel in the last release
Dashboard empty	Wrong time range, data source or variable	Query in the explorer first
Monitoring slow or expensive	High-cardinality labels (user id, request id)	Series count by metric
No alert fired	Rule never evaluated, or notification route missing	Rule state, alert manager routing

Alert on the monitoring itself: the `up` metric, the age of the newest log line, and a regular test alert (a "dead man's switch") that must always fire.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: From alert to cause

Part	What it does	What goes wrong
Alert fires	A symptom alert on errors or latency starts the investigation.	Cause-based alerts with no user impact waste attention.
Metrics: what and when	Split by route, version and region to find the affected slice.	Looking at overall averages hides small broken parts.
Traces: where	Find the slow or failing span in one request.	Missing context propagation breaks traces at service borders.
Logs: why	The trace id leads to the log line with the error text.	Logs without ids force time-based guessing.
Act and verify	Roll back or fix, and watch the same metric recover.	No verification leaves doubt that the issue is closed.

Flow: Alert fires to Metrics: what and when / Metrics: what and when to Traces: where / Traces: where to Logs: why / Logs: why to Act and verify

Incident: the dashboard that said everything was fine

Incident: *Customers report failed checkouts for 40 minutes while every dashboard stays green.*

```
$ curl -s localhost:9090/api/v1/query --data-urlencode 'query=up{job="checkout"}' | jq '.data.resu \
    lt | length'
0
```

A relabelling change removed the `job="checkout"` target, so the metrics stopped, and a rule based on "error rate above 2 percent" saw no data and stayed quiet. The team added an alert for missing data, an alert on the number of targets up, and a synthetic checkout probe from outside.

Common mistakes

- Alerting only on causes such as CPU
- Using averages for latency
- Putting user or request ids in metric labels
- Trusting an empty graph to mean "no errors"

Check yourself

Which signal best tells you where in a request path the time is spent?

Answer: A distributed trace

What does a trace id in log lines allow?

Answer: Jumping from a span to the exact related log entries

A graph shows no data and no alert fired. What should you suspect?

Answer: The target or pipeline is broken, so alert on missing data

Why are request ids bad as metric labels?

Answer: Each value creates a new series, causing very high cardinality and cost

MARGA by EknathaLabs marga.eknathalabs.com